REQUERIMIENTOS NO FUNCIONALES (RNF)

RNF-01. Stack tecnológico definido
El backend debe implementarse sobre Node.js 20 o superior, Express 4, Sequelize 6 como ORM y MySQL 8 como base de datos de producción. La autenticación se basará en JWT y el hash de contraseñas en Bcrypt con un mínimo de 10 rondas. El frontend debe usar React 18 con TypeScript, Vite, Tailwind CSS, TanStack Query, Zustand, React Hook Form con Zod y React Router 6.

RNF-02. Arquitectura en capas
El backend debe seguir un flujo claro de responsabilidades: rutas → validadores y middlewares → controladores → servicios → modelos. Las reglas de negocio y las transacciones deben residir en la capa de servicios, manteniendo los controladores como adaptadores HTTP delgados.

RNF-03. Seguridad
El sistema debe aplicar control de acceso basado en roles (RBAC) en todas las rutas protegidas, validación de entrada con express-validator, rate limiting global y específico para autenticación, y configuración de CORS restringida por origen. En producción será obligatorio reemplazar el JWT_SECRET y las contraseñas de ejemplo, así como operar bajo HTTPS.

RNF-04. Integridad y concurrencia
Las operaciones críticas (inscripciones y entrega de kits) deben ejecutarse dentro de transacciones con bloqueo pesimista (SELECT … FOR UPDATE en MySQL) para garantizar la consistencia ante accesos concurrentes. Se deben definir restricciones únicas a nivel de base de datos: (id_usuario, id_categoria) en inscripciones y (id_dorsal, nombre_punto) en tiempos de checkpoint, además de unicidad global en dorsales.numero_dorsal.

RNF-05. Rendimiento y concurrencia
El sistema debe soportar picos de hasta 300 solicitudes concurrentes por segundo. El rate limit por defecto debe permitir ráfagas de hasta ~500 req/s por IP (30 000/min), ajustable según el tráfico real y las políticas de abuso.

RNF-06. Portabilidad de base de datos para pruebas
La suite de pruebas debe ejecutarse sobre SQLite en memoria, sin dependencia de MySQL ni Docker. El dialecto de producción será MySQL, donde se aplicarán tipos ENUM, DECIMAL, índices y bloqueos. Esta separación debe mantenerse mediante la variable DB_DIALECT.

RNF-07. Calidad y cobertura de pruebas
El proyecto debe incluir una suite de integración con node:test y supertest que cubra el flujo integral: autenticación, tema, RBAC, eventos y categorías, cupos, inscripciones, pagos, kits, dorsales, checkpoints, tiempo neto, resultados, certificado, estadísticas y errores estandarizados.

RNF-08. Configurabilidad por entorno
Toda la configuración sensible debe gestionarse mediante variables de entorno: conexión a base de datos, secreto JWT, expiración del token, rondas de Bcrypt, orígenes CORS, límites de rate limiting, credenciales del administrador inicial y banderas como DB_SYNC y SEED_ON_START. En producción, el seeder no debe ejecutarse al iniciar y DB_SYNC debe desactivarse en favor de migraciones controladas.

RNF-09. Despliegue y operación
El backend debe poder levantarse localmente con Docker Compose para MySQL 8, y el frontend debe compilarse con npm run build (incluyendo typecheck con tsc --noEmit) para servirse como estáticos en Nginx, Vercel, Netlify o similar, con rewrite de SPA hacia index.html y proxy de /api hacia el backend.

RNF-10. Observabilidad
Los errores 5xx deben registrarse en consola con capacidad de integración a un colector de logs en producción. El sistema debe exponer un endpoint /health para verificación del estado del servicio.

RNF-11. Usabilidad y experiencia de usuario
La interfaz debe ser responsive, con tema oscuro por defecto (paleta slate-950 con acentos verde neón y naranja), detección de preferencia del sistema y toggle manual persistido en localStorage. La API debe respetar la preferencia tema.modo_oscuro del usuario autenticado y devolver false en rutas públicas o sin token.

RNF-12. Mantenibilidad y documentación
El código debe mantenerse organizado por dominios (config, models, controllers, services, routes, middlewares, validators, utils, docs, seeders, scripts). La documentación debe incluir README con inicio rápido, scripts, modelo de datos, matriz RBAC, endpoints principales, reglas de negocio, variables de entorno y notas de producción, además de la especificación OpenAPI.

RNF-13. Escalabilidad del frontend
El frontend debe gestionar el estado del servidor con TanStack Query y el estado global con Zustand, aplicando validación de formularios con Zod y React Hook Form. La generación de

PDF (jsPDF), exportaciones (xlsx) y escaneo QR (html5-qrcode) deben ejecutarse del lado del cliente para no sobrecargar el backend.